



COMPSCI 130

Introduction to Software Fundamentals

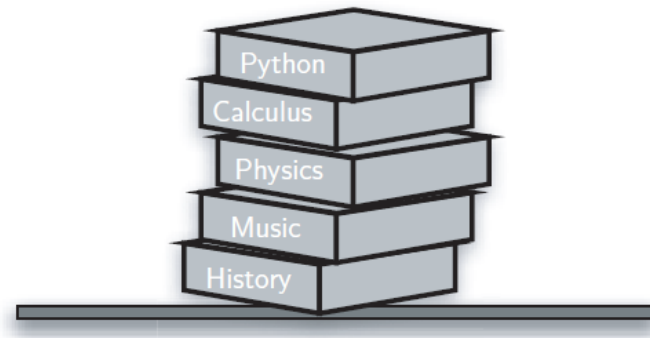
ADTS and Stacks



Agenda & Reading

► Agenda

- Abstract Data Type (ADT)
- The Stack Abstract Data Type (ADT)
- Two Implementations
- Stack Applications
 - Checking for Balanced Braces
 - Bracket Matching
 - Postfix Calculation
 - Conversion from Infix to Postfix



► Reading

- Chapter 13 of the Online Textbook



Abstract Data Types

- ▶ An ADT is composed of
 - ▶ A collection of **data**
 - ▶ A set of **operations** on that data
- ▶ Specifications of an ADT indicate
 - ▶ What the ADT operations do, not how to implement them
- ▶ Implementation of an ADT
 - ▶ Includes choosing a particular data structure
- ▶ ADT is a logical description of how we view the data and the operations that are allowed without regard to how they will be implemented



Linear Structures

- ▶ Linear structures are data collections whose items are **ordered** depending on how they are **added** or **removed** from the structure.
- ▶ Once an item is added, it stays in that **position** relative to the other elements that came before and came after it.
- ▶ Linear structures can be thought of as having two **ends**, **top** and **bottom**, (or front and end or front and back)
- ▶ What distinguishes one linear structure from another is the way in which **items are added and removed**, in particular the **location** where these **additions** and **removals occur**, e.g., add only to one end, add to both, etc.

What is a Stack?

- ▶ A stack is an ordered collection of items where the **addition** of new items and the **removal** of existing items always takes place at the same end, referred to as the **top** of the stack.
 - ▶ i.e. add at top, remove from top
- ▶ Last-in, first-out (LIFO) property
 - ▶ The last item placed on the stack will be the first item removed
- ▶ Example:
 - ▶ A stack of dishes in a cafeteria

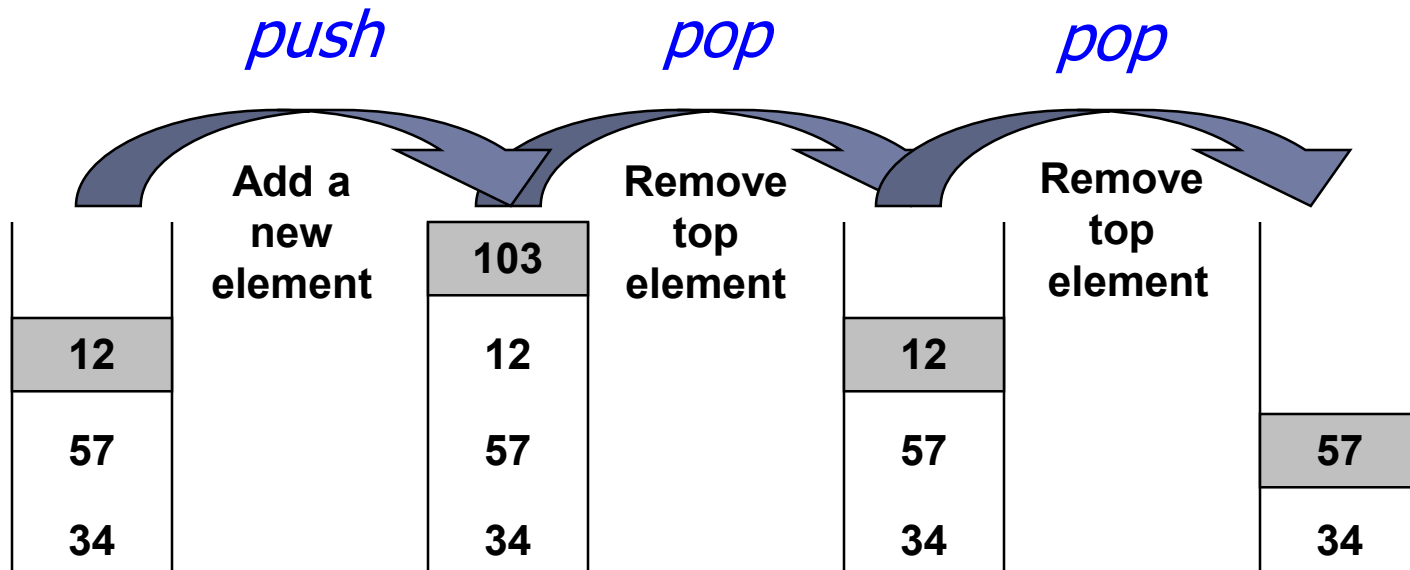




An Example

- ▶ Add only to the **top** of a Stack
- ▶ Remove only from the **top** of the Stack
 - ▶ Note: The last item placed on the stack will be the first item removed

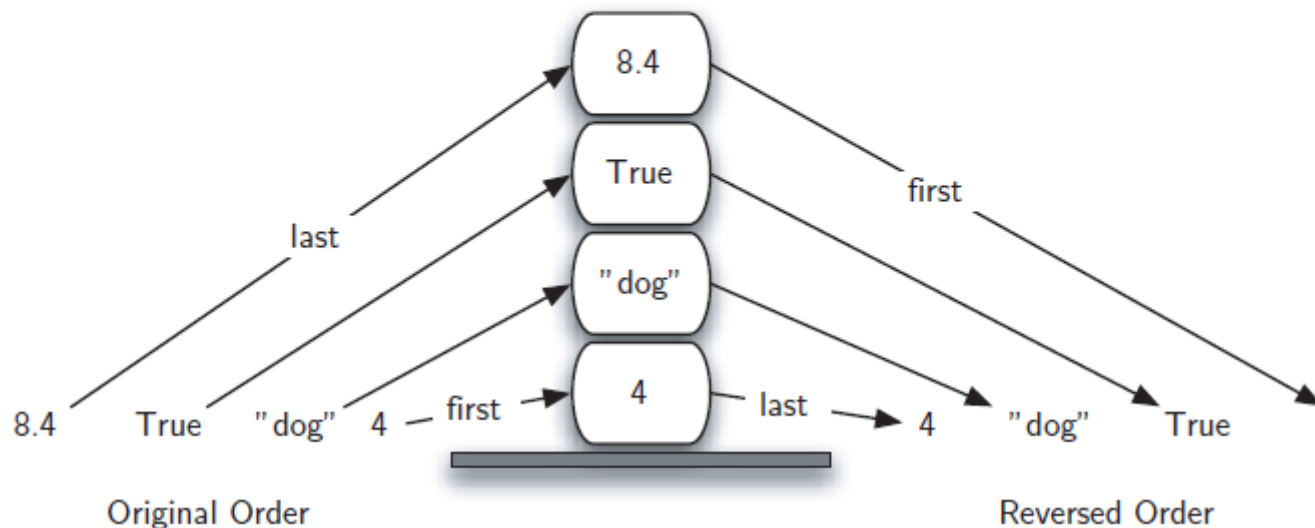
Last In - First Out (LIFO)



Orders

- ▶ The **base** of the stack contains the **oldest** item, the one which has been there the **longest**.
- ▶ For a stack the order in which items are removed is exactly the **reverse of the order** that they were placed.

The Reversal Property of Stacks





ADT Stack Operations

- ▶ What are the operations which can be used with a Stack Abstract Data?
 - ▶ Create an **empty** stack
 - ▶ Determine whether a stack is **empty**
 - ▶ Add a new item to the stack
 - ▶ **push**
 - ▶ Remove from the stack the item that was added most recently
 - ▶ **pop**
 - ▶ Retrieve from the stack the item that was added most recently
 - ▶ **peek**



The Stack Abstract Data Type

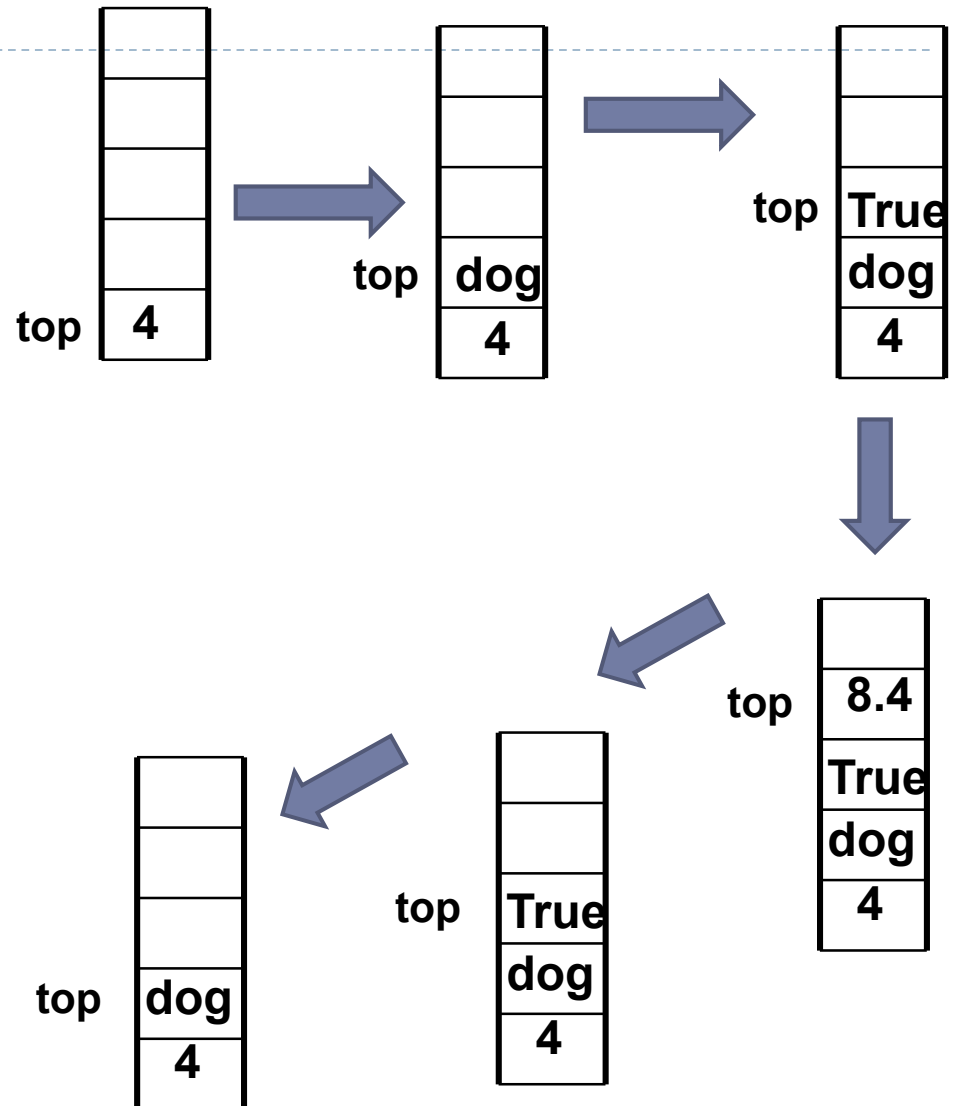
- ▶ Stack() creates a new stack that is empty.
 - ▶ It needs no parameters and returns an empty stack.
- ▶ push(item) adds a new item to the top of the stack.
 - ▶ It needs the item and returns nothing.
- ▶ pop() removes the top item from the stack.
 - ▶ It needs no parameters and returns the item. The stack is modified.
- ▶ peek() returns the top item from the stack but does not remove it.
 - ▶ It needs no parameters. The stack is **not** modified.
- ▶ is_empty() tests to see whether the stack is empty.
 - ▶ It needs no parameters and returns a boolean value.
- ▶ size() returns the number of items on the stack.
 - ▶ It needs no parameters and returns an integer.

Code Example

► Code:

```
from Stack import Stack

s = Stack()
print(s.is_empty())
s.push(4)
s.push('dog')
print(s.peek())
s.push(True)
print(s.size())
print(s.is_empty())
s.push(8.4)
s.pop()
s.pop()
print(s.size())
```





Code Example

- ▶ Stack operations, state of the stack, values returned

```
s.is_empty()
s.push(4)
s.push('dog')
s.peek()
s.push(True)
s.size()
s.is_empty()
s.push(8.4)
s.pop()
s.pop()
s.size()
```

```
[]
[4]
[4,'dog']
[4,'dog']
[4,'dog',True]
[4,'dog',True]
[4,'dog',True]
[4,'dog',True,8.4]
[4,'dog',True]
[4,'dog']
[4,'dog']
```

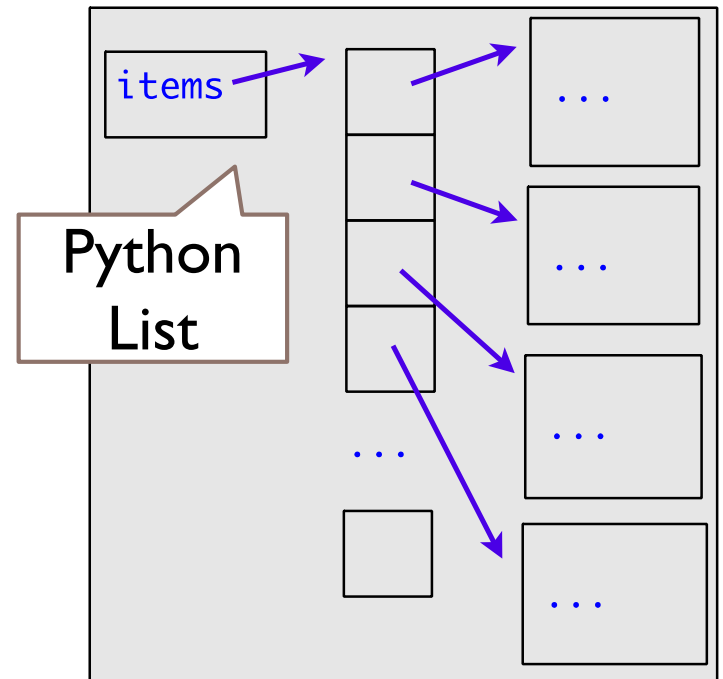
Top

```
True
'dog'
3
False
8.4
True
2
```

Stack Implementation in Python

- ▶ We use a python List data structure to implement the stack.
 - ▶ Remember: the addition of new items and the removal of existing items always takes place at the same **end**, referred to as the **top** of the stack.
 - ▶ But which “end” is better?

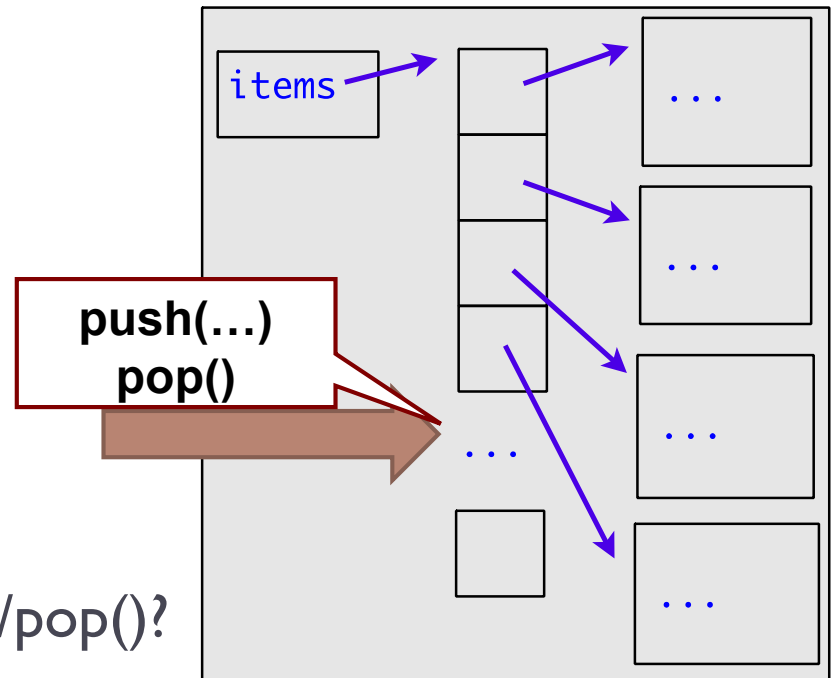
```
class Stack:  
    def __init__(self):  
        self.items = []  
    def is_empty(self):  
        return self.items == []  
    def size(self):  
        return len(self.items)  
  
    def push(self, item) ...  
    def pop(self) ...  
    def peek(self) ...
```



Version 1

- ▶ This implementation assumes that the end of the list will hold the top element of the stack.
- ▶ As the stack grows, new items will be added on the **END** of the list. Pop operations will manipulate the same end

```
class Stack:  
    ...  
    def push(self, item):  
        self.items.append(item)  
  
    def pop(self):  
        return self.items.pop()  
    ...
```

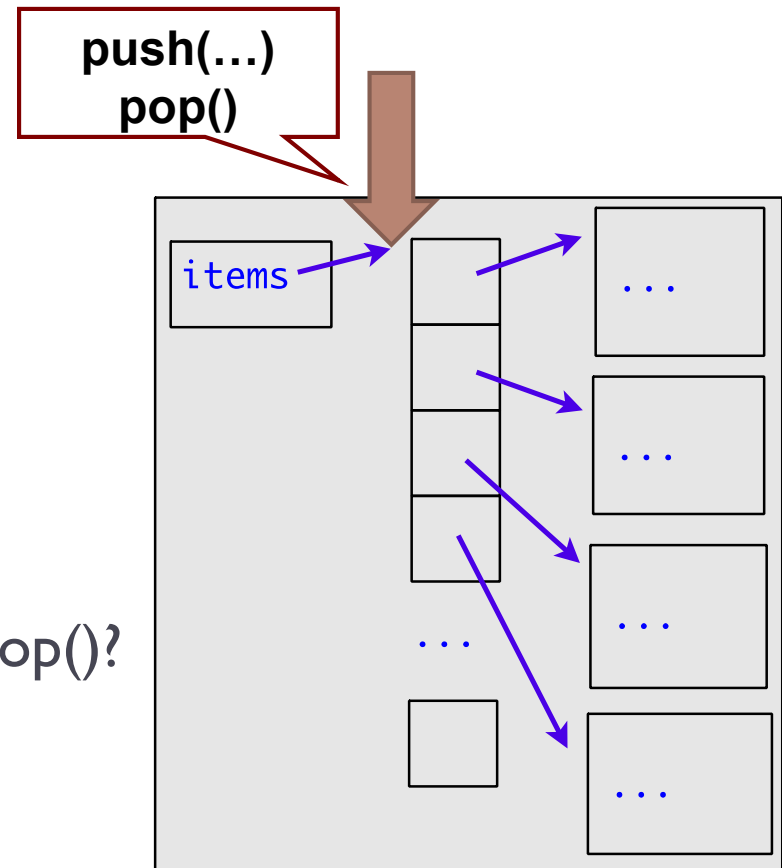


- ▶ What is the Big-O of the push()/pop()?

O(1)

- ▶ This implementation assumes that the **beginning** of the list will hold the top element of the stack.

```
class Stack:  
    ...  
    def push(self, item):  
        self.items.insert(0,item)  
  
    def pop(self):  
        return self.items.pop(0)  
    ...
```



- ▶ What is the Big-O of the push()/pop()?

$O(n)$



Exercise

Consider the Stack ADT implemented using a Python List (called `items`) such that `push()` and `pop()` are defined as follows:

```
def push(self, item):  
    self.items.append(item)  
  
def pop(self):  
    return self.items.pop()
```

Which of the following statements is TRUE?

- (a) both `push()` and `pop()` are $O(n)$
- (b) `push()` is $O(1)$ and `pop()` is $O(n)$
- (c) both `push()` and `pop()` are $O(n^2)$
- (d) `pop()` is $O(1)$ but `push()` is $O(n)$
- (e) both `push()` and `pop()` are $O(1)$



Stack Applications: Checking for Balanced Braces

- ▶ Using a stack to verify whether braces are balanced
 - ▶ An example of **balanced** braces
 - ▶ { { } { { } } }
 - ▶ An example of **unbalanced** braces
 - ▶ { } } { { }
- ▶ Requirements for balanced braces
 - ▶ Each time you encounter a “}”, it matches an already encountered “{”
 - ▶ When you reach the end of the string, you have matched each “{”



Stack Applications: Balanced Braces - Algorithm

► Steps:

Initialise the stack to empty

For every char read

- If it is an open bracket/brace, then push onto stack
- If it is a close bracket/brace, then
 - If the stack is empty, return ERROR
 - Pop an element out from the stack
 - If the bracket/brace at the top of the stack does not match the current char then return ERROR
- if it is a non-bracket character, skip it

If the stack is NON-EMPTY, ERROR

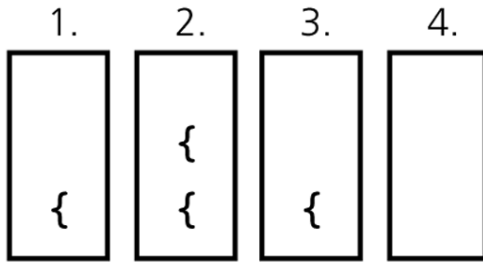


Stack Applications: Balanced Braces - Example

Input string

Stack as algorithm executes

{a{b}c}



1. push "{"

2. push "{"

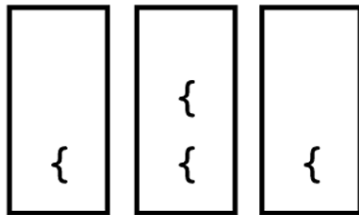
3. pop

4. pop

Stack empty $\Rightarrow$ balanced



{a{bc}



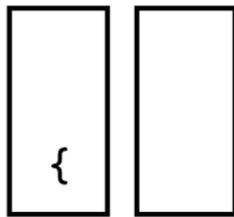
1. push "{"

2. push "{"

3. pop

Stack not empty $\Rightarrow$ not balanced

{ab}c}



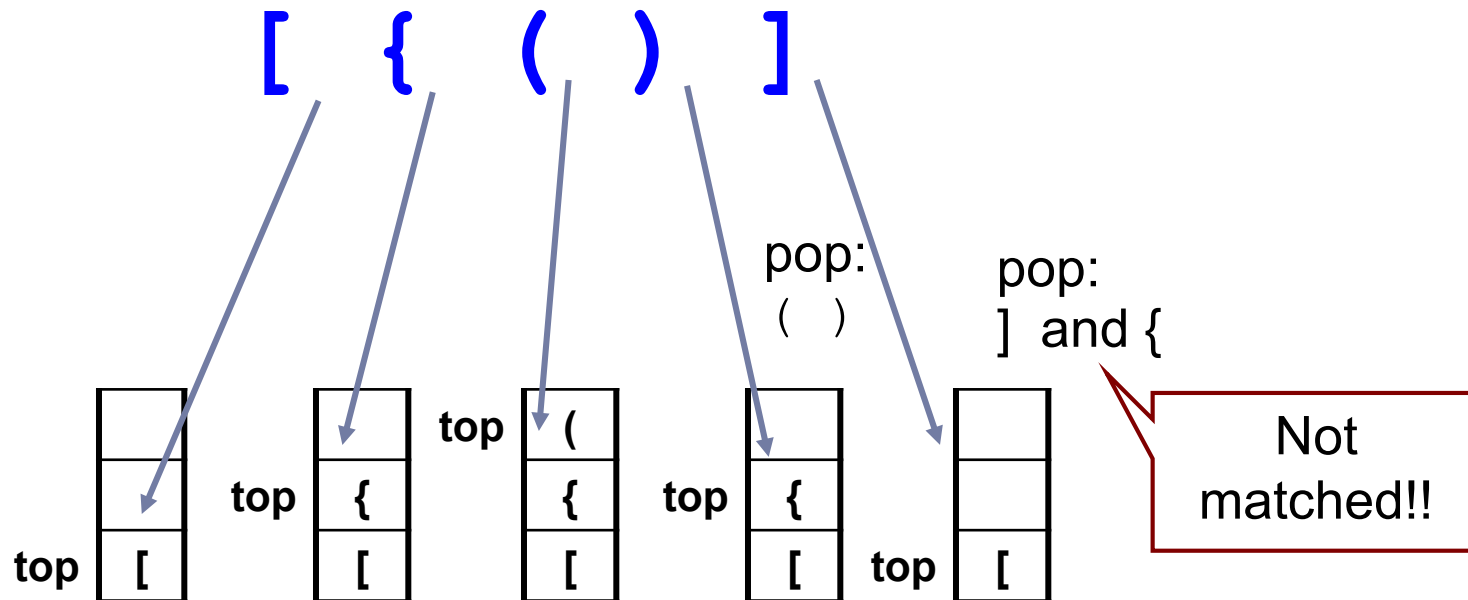
1. push "{"

2. pop

Stack empty when last "}" encountered $\Rightarrow$ not balanced



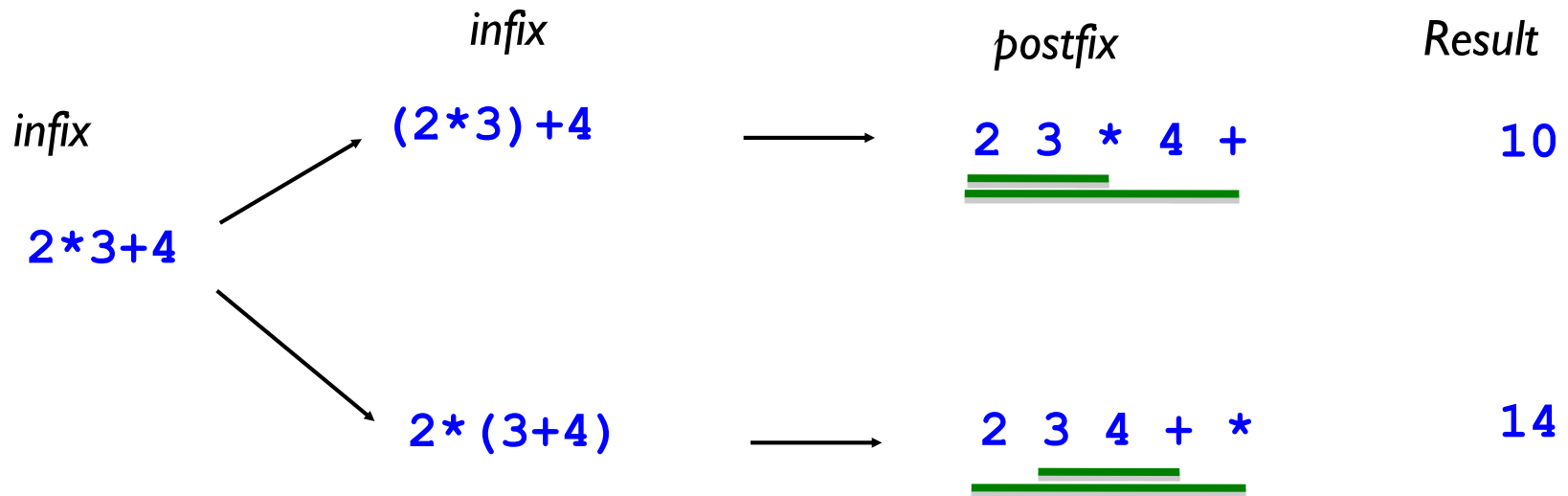
Stack Applications: Balanced Braces - Example





Stack Applications: Postfix Calculator

- ▶ Computation of arithmetic expressions can be efficiently carried out in Postfix notation with the help of stack
 - ▶ Infix - arg1 op arg2
 - ▶ Prefix - op arg1 arg2
 - ▶ Postfix - arg1 arg2 op





Stack Applications: Postfix Calculator - Algorithm

- ▶ Requires you to enter postfix expressions
 - ▶ Example: 2 3 4 + *
- ▶ Steps:

When an operand is reached,

- **Push the operand onto the stack**

When an operator is reached,

- **Pop the top two operands from the stack**
- **Perform the arithmetic using these operands**
- **Push the result of the operation on the stack**



Stack Applications: Evaluating the expression: 2 3 4 + *

Postfix Calculator – Example 1

Key entered	Calculator action	Stack (bottom to top)
2	push 2	2
3	push 3	2 3
4	push 4	2 3 4
+	operand2 = pop stack (4)	2 3
	operand1 = pop stack (3)	2
	result = operand1 + operand2 (7)	2
	push result	2 7
*	operand2 = pop stack (7)	2
	operand1 = pop stack (2)	
	result = operand1 * operand2 (14)	
	push result	14



Stack Applications: Evaluating the expression: $2\ 3\ *\ 4\ +$

Postfix Calculator – Example 2

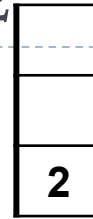
key entered

Calculator action

2

`s.push (2)`

top



3

`s.push (3)`

top



*

`arg2 = s.pop()`

`arg1 = s.pop()`

`s.push(arg1 * arg2)`

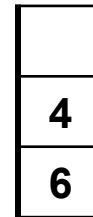
top



4

`s.push (4)`

top



+

`arg2 = s.pop ()`

`arg1 = s.pop ()`

`s.push (arg1 + arg2)`

top





Exercise

What is the result of evaluating the following postfix expression?

$$2\ 3\ 1\ -\ 2\ *\ +\ 5\ -$$



Stack Applications:

Conversion from Infix to Postfix

► Examples:

► $2 * 3 + 4 \Rightarrow$

2 3 * 4 +

► $2 + 3 * 4 \Rightarrow$

2 3 4 * +

► $1 * 3 + 2 * 4 \Rightarrow$

1 3 * 2 4 * +

► Steps:

- **Operands** always stay in the same order with respect to one another
- An **operator** will move only “to the right” with respect to the operands
- All parentheses are removed



Stack Applications:

Conversion from Infix to Postfix - Algorithm

When we reach an operand

- Just append the operand to the output string

When we reach an open parenthesis: “(”

- Push the “(” to the stack

When we reach an operator

- If the stack is empty, then push the operator onto the stack
- If the stack is non-empty, then:
 - Pop the operators of greater or equal precedence from the stack and append them to the output string
 - Stop when we encounter either: a “(”, an operator of lower precedence, or when the stack is empty
 - Push the new operator onto the stack

When we reach a “)”

- Pop the operators off the stack and append them to the end of the output string until we encounter the matching “(” which we also pop

When we reach the end of the input string

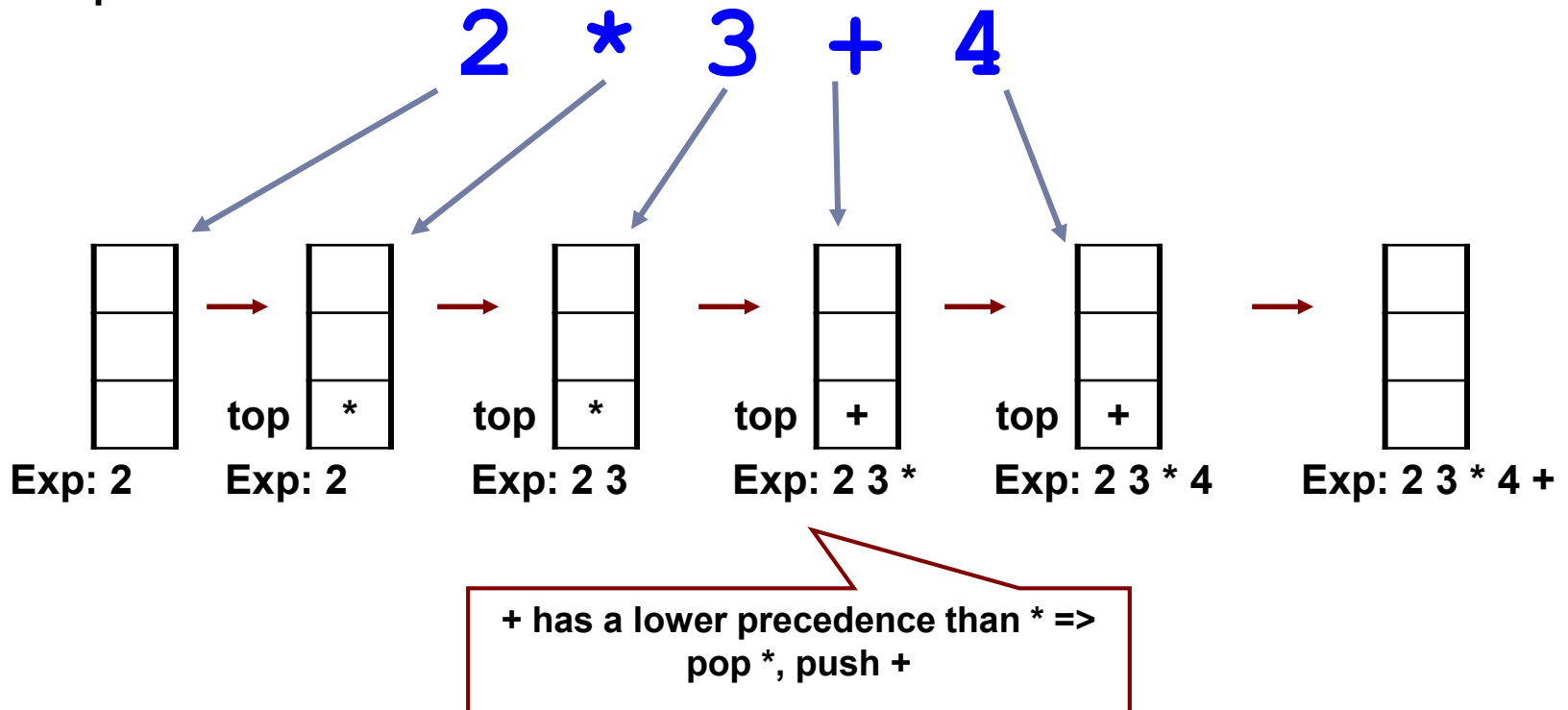
- Pop the remaining contents off the stack and append it to the output string



Stack Applications:

Examples - Converting Infix to Postfix

► Example I

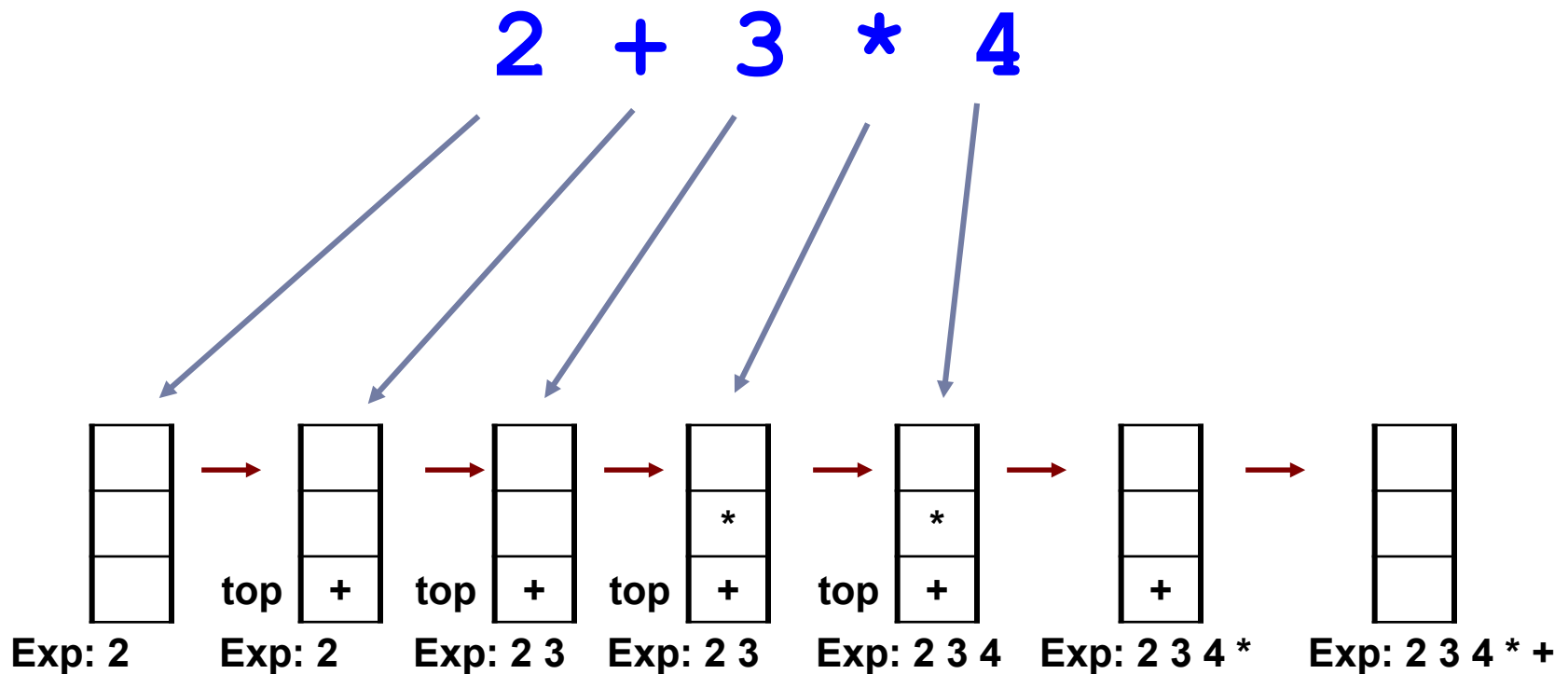




Stack Applications:

Examples - Converting Infix to Postfix

► Example 2:





Exercise

What is the postfix notation for the following infix expression?

$$(7 + 2) / (9 + 1)$$